CE2003 Tut 7

Q1 The following Verilog code fragment represents a computationally intensive block used many times. It needs to be accelerated. Assume that there are only two input, one output arithmetic blocks are available, and the multiplier and the adder blocks have the same computational latency (T).

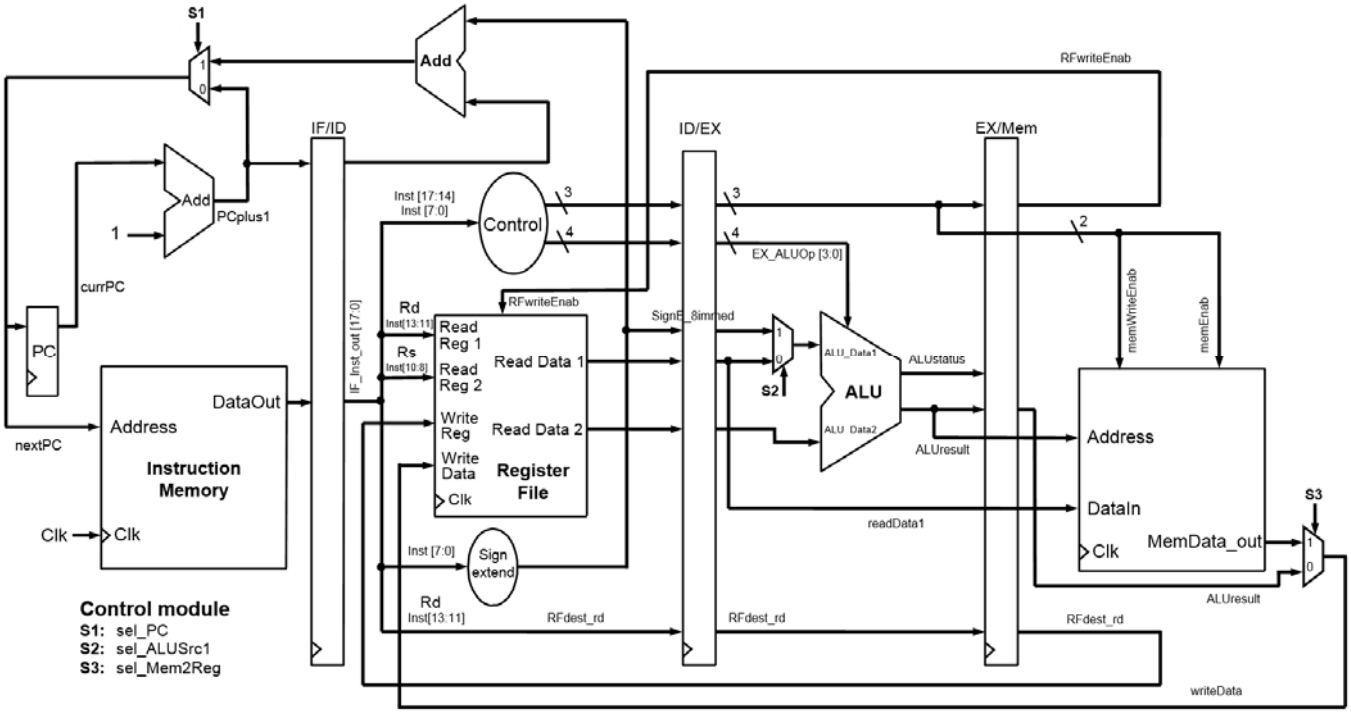
```
begin
  X <= E*(A+B+C);
  Y <= (A+C)*(C+D);
end
```

a. Determine the *as late as possible* (ALAP) schedule.
b. Determine the best throughput, the latency and the computational resource needed.
c. Pipeline the computations, using an *as soon as possible* (ASAP) implementation.
d. Determine the best throughput, the latency and the computational resource needed.

Q2 Repeat Q1, but now assume that the multiplier has twice the computational latency of the adder (i.e. 2*T* seconds).

Q3 Repeat Q1, but now assume that the multiplier has twice the computational latency of the adder (i.e. 2T seconds) and is internally pipelined (with 2 pipeline stages). Note data can be input and output on each clock cycle.

Q4 A simple pipelined processor is implemented in Verilog as per the attached block diagram. The processor has an 18 bit instruction format, while the datapath and data memory are 16 bit. Describe the action performed in each stage of the processor, and the number of clock cycles from the point where a new PC value is latched into the PC register to the point where a data value is written back into the register file. How many stages does the processor have?

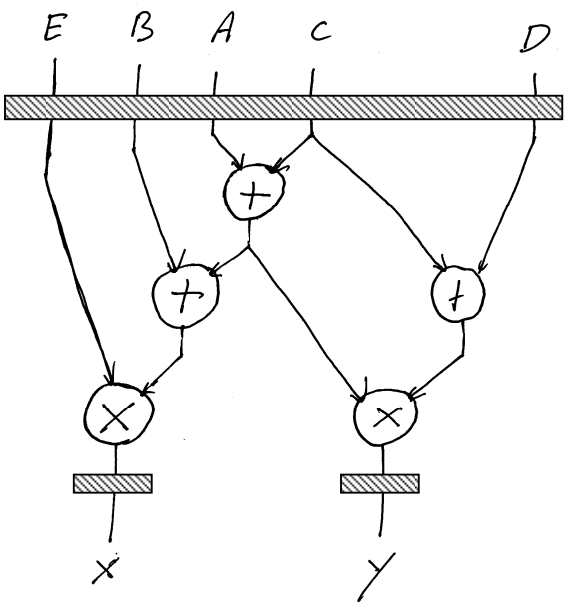


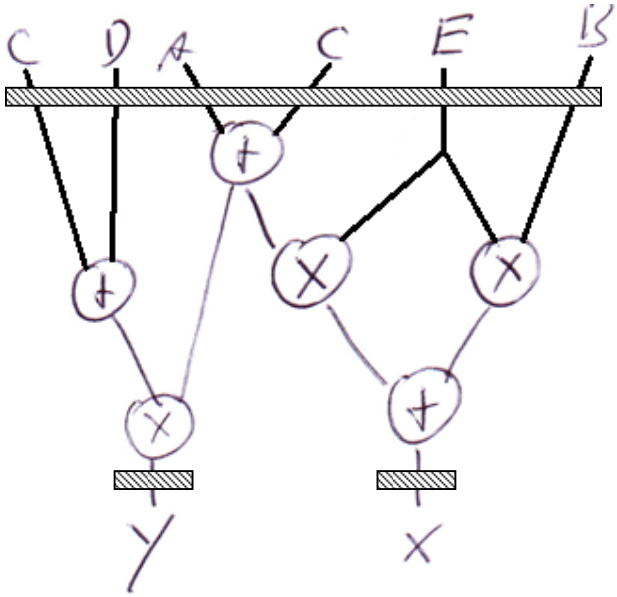
Q1 (a) You could rearrange the expression as one of two ways:

X <= E*(B+(A+C)); X <= E*B+E*(A+C));

Y <= (C+D)*(A+C); Y <= (C+D)*(A+C);

ALAP schedule (with registers added at the inputs and outputs) is:

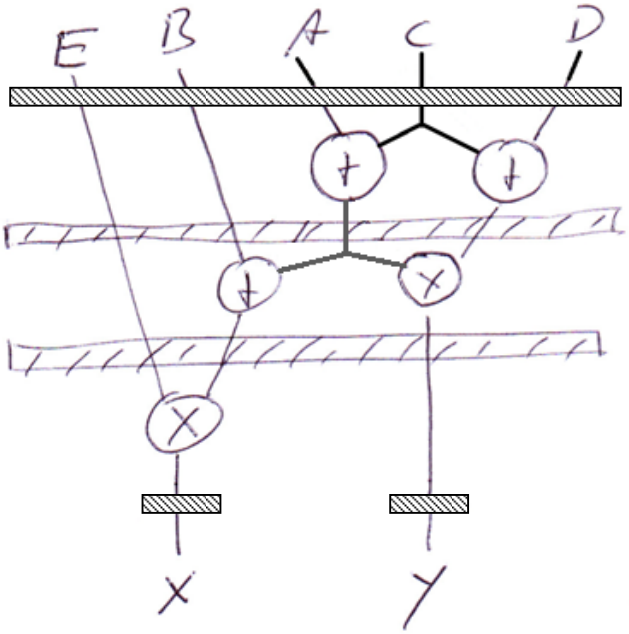




(b) Thus the first option is better as it has:

A throughput of one every 3T; a latency of 3T; and requires 3 adders and 2 multipliers. Option 2 has same throughput and latency, but requires 3 adders and 3 multipliers.

(c) We can pipeline. Option 1 would still be best. The ASAP schedule is:



(d) It has a throughput of one every T; a latency of 3T; and requires 3 adders, 2 multipliers and 7 registers, not including the 5 input registers and the 2 output registers.

Q2(a) Again, the first expression (option 1) will produce the best result:

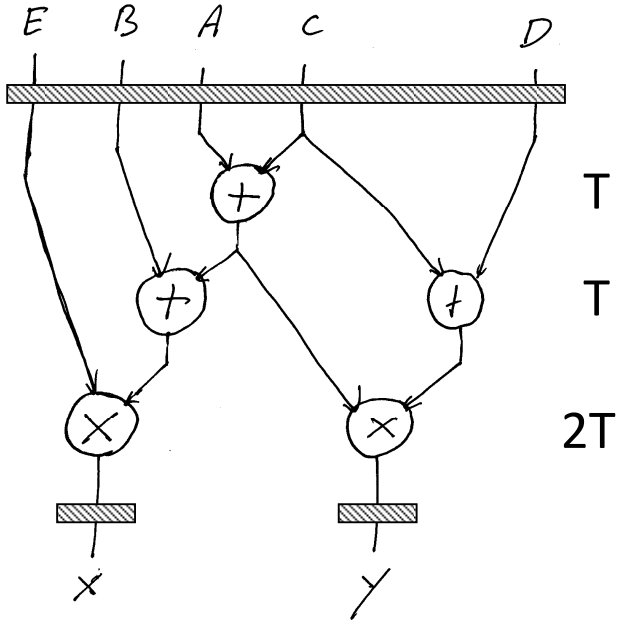
X <= E*(B+(A+C));

Y <= (C+D)*(A+C);

~~X <= E*B+E*(A+C));~~

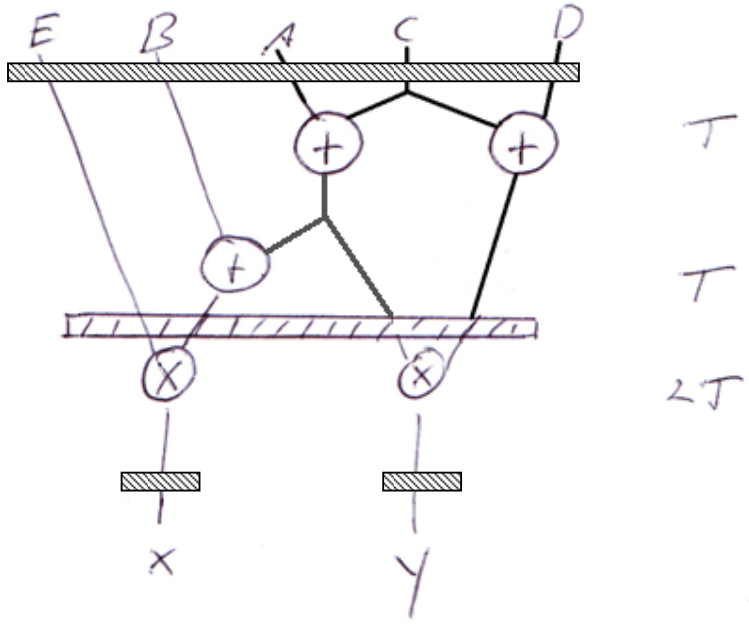
~~Y <= (C+D)*(A+C);~~

ALAP schedule is:



(b) This option has a throughput of one every 4T; a latency of 4T; and requires 3 adders and 2 multipliers

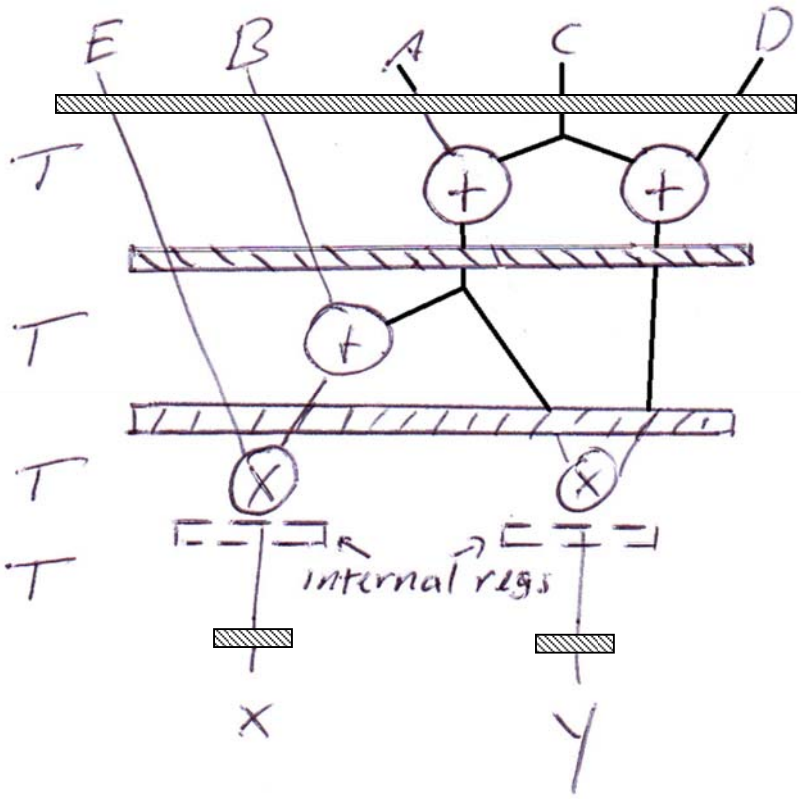
(c) We can pipeline. Option 1 would still be best. Balancing the pipeline stages, results in a 2 stage pipelined implementation. The ASAP schedule is:



(d) It has a throughput of one every 2T; a latency of 4 T; and requires 3 adders, 2 multipliers and 4 registers (not including the 7 I/O registers).

Q3 (a) (b) You would not operate this circuit in a non-pipelined way. Hence, (a) and (b) should be ignored.

(c) We should pipeline. Again option 1 would still be best. Balancing the pipeline stages, results in a 4 stage pipelined implementation. The ASAP schedule is:



(d) It has a throughput of one every T; a latency of 4T; and requires 3 adders, 2 multipliers and 8 registers, not including the I/O registers.

Note: all registers above are of a size equal to the internal word size.

Q4: On the 1st rising clock edge, the next PC value is latched into the PC register. At the same time, the next PC value is latched into the instruction memory (which is clocked, e.g. synchronous). After the memory access time, the instruction pointed to by the PC will appear at the memory DataOut port. This is cycle 1, the IF stage.

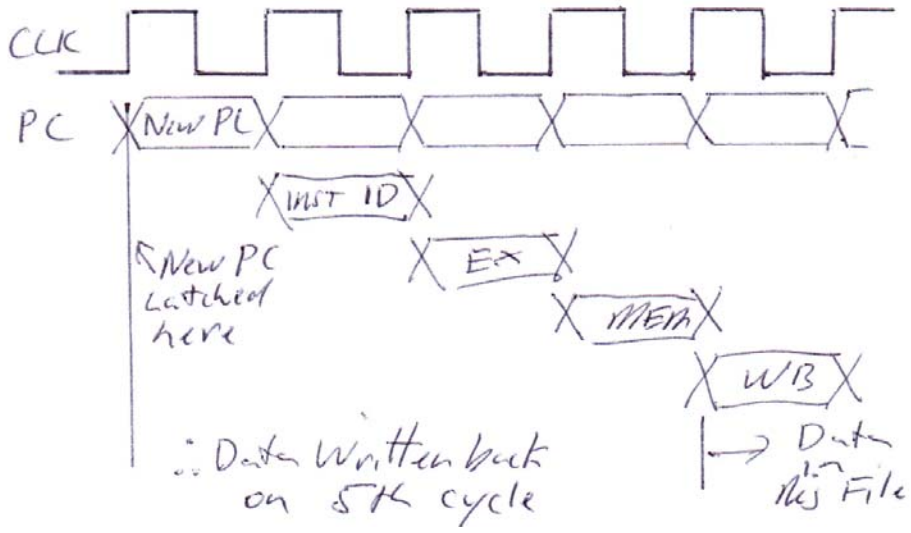
On the 2nd rising edge, the instruction is latched in the IF/ID pipeline register. This is split into parts and fed to the control, register file (RF), etc. The RF access is combinational. After the RF access time, the values from the RF pointed to by rs & rd will appear on the ReadData1 & ReadData2 RF outputs. This is cycle 2, the ID stage.

On the 3rd rising edge, these values will be latched into the ID/EX pipeline register. The ALU will perform an operation on these values and after a delay will produce a result which will appear on the ALUresult output. This is cycle 3, the EX stage.

On the 4th rising edge, the ALUresult is latched by the EX/Mem pipeline register. At the same time, the ALUresult is latched into the data memory address (The data memory is also clocked, e.g. it is synchronous). After the memory access time, the data will appear at the MemData_Out port of the data memory. Mux S3 then selects either the ALU result or the Memory data, depending on the instruction. This is cycle 4, the Mem stage

On the 5th rising edge, the output of the S3 mux is written back to the register file, e.g. RF write back is clocked. This write back to the RF occurs after the RF clock-to-Q delay, so write back is will obviously occur during the 5th cycle. This is cycle 5, the WB stage.

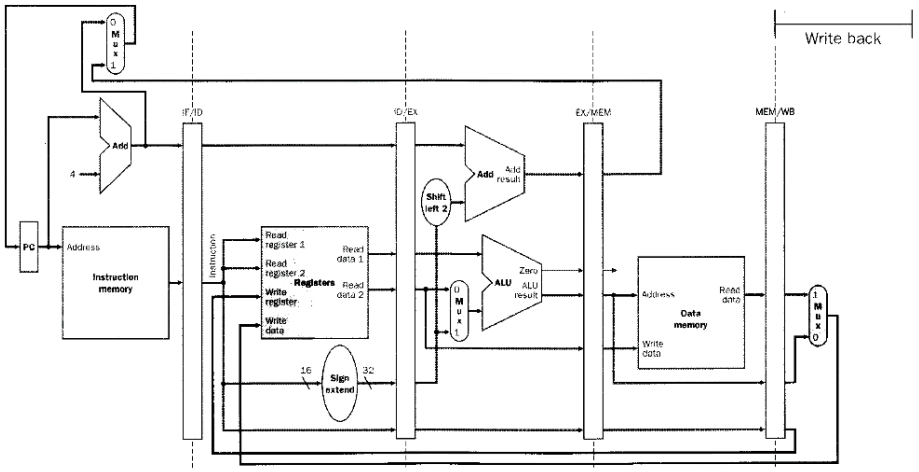
Graphically, each of these steps is shown below.



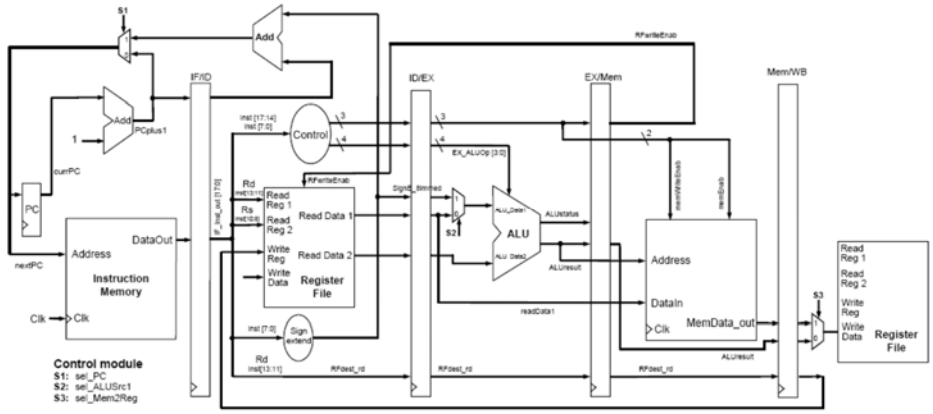
Therefore, data is written back just after the 5th rising clock edge. That is at the start of the 5th cycle.

Thus the processor has 5 stages and hence it is a five stage pipelined processor. To show this, consider the 5 stage MIPS like processor for Patterson and Hennessy. It obviously has 5 stages.

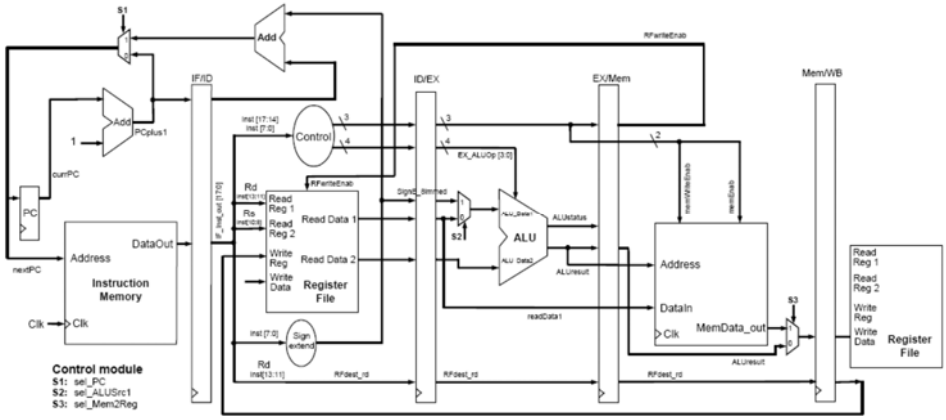
Note the RF is not clocked and instead has a combinational write and memory is not clocked



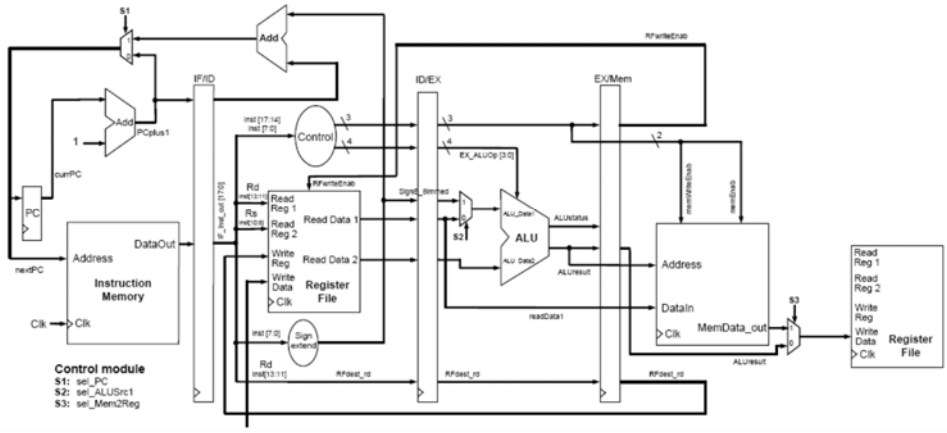
Let's replicate the register file to the WB stage, to make the analysis simpler. The WB stage looks the same as the Patterson and Hennessy MIPS diagram above. Note we have also changed the memory to clocked.



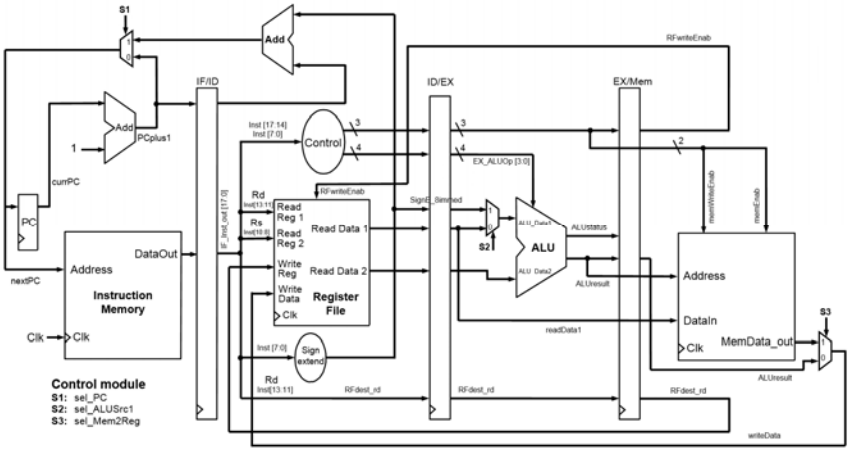
Then move the S3 mux to the mem stage. This makes no difference.



Then merge the pipeline register into the register file. That is make the write back clocked, rather that combinational.



Then if we put the register file back where it should be, then we now have the uP16 implementation



So… Same number of stages and same number of cycles

That is 5!